

U.S. Flight Data Analysis with Apache Spark

A Batch Processing Pipeline on a Containerized Architecture

Systems and Architectures for Big Data, a.y. 2025/26

Valentina Jin
Computer Science Engineering
Tor Vergata University of Rome
valentina.jin@students.uniroma2.eu

Elisa Marzioli
Computer Science Engineering
Tor Vergata University of Rome
elisa.marzioli@students.uniroma2.eu

Abstract—This project presents a batch processing pipeline for the analysis of ~2.2 million records of US flights (January-April 2025), offered by the Bureau of Transportation Systems of the United States of America. The architecture, containerized via Docker Compose, orchestrates thanks to Apache Airflow the following components: Apache NiFi, HDFS, Apache Spark (PySpark), Redis e Grafana. Three analytical queries are performed on delays and flight cancellations. The goal of the study is that of analyzing the results of the queries and determining the impact of the configuration parameters on the performance of the batch processing pipeline.

Key words—Spark, PySpark, HDFS, Docker, NiFi, Redis, Grafana, Airflow, batch processing, big data.

I. INTRODUCTION

The analysis of large volumes of data in the transportation sector represents an emblematic use case for Big Data processing technologies. In this context, the project aims to build a complete pipeline that, starting from raw flight data, makes it possible to extract meaningful information on delays, cancellations, and their causes through three analytical queries implemented with Apache Spark [1]. The dataset, provided by the U.S. Department of Transportation (BTS) [2], covers the period January-April 2025 and includes information on airlines, schedules, departure and arrival delays, flight status (cancelled/diverted), and causes of delay. The choice of technologies reflects an approach oriented toward scalability and reproducibility: Docker Compose [3] for containerization, Apache Airflow [4] for orchestration, Apache NiFi [5] as the data ingestion tool, Parquet as the intermediate storage format on HDFS [6], Redis [7] as the database for storing the analytical results, and Grafana [8] for visualizing the results.

II. SYSTEM ARCHITECTURE

The architecture is fully containerized using Docker Compose and consists of six main components orchestrated by Apache Airflow using the `sabd_project1_pipeline` DAG.

Fig. 1 illustrates the data flow and the interactions between the components.

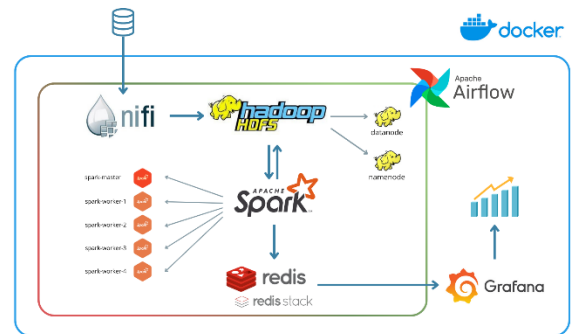


Figure 1. Scheme of the distributed system's architecture

The main components are:

- **Apache NiFi:** handles automated data ingestion. It downloads the monthly datasets from the URL in .tar.gz format, decompresses them, and loads them onto HDFS under the path `/data/raw/flights/csv/`, acting as a controlled entry point for the raw data into the pipeline.
- **Hadoop Distributed File System (HDFS):** serves as persistent distributed storage for the raw data, the processed data converted to Parquet format, and the output. In the current configuration it consists of one datanode and one namenode, given the local deployment.
- **Apache Spark:** constitutes the computational core of the system. It runs the preprocessing and the three analytical queries through PySpark, leveraging a cluster in Standalone mode with one master and a variable number of workers, from 1 to 4. The resources for each Spark Worker are limited to 1GB of RAM and 1 core, to allow a scalability analysis even under local deployment conditions.
- **Redis:** serves as the serving layer between Spark and Grafana. The aggregated results produced by the queries are exported to Redis Stack [9] to make the data directly accessible to the Grafana panels.
- **Grafana:** open-source platform for interactive data visualization. It accesses Redis through the Redis Data Source plugin and provides a dedicated dashboard for each query, with time-series panels, bar charts, and tables

- **Apache Airflow:** end-to-end orchestrator of the pipeline. The DAG `sabd_project1_pipeline` defines the dependencies between tasks, ensuring correct sequencing (ingestion → preprocessing → query → export) and execution monitoring.
- **Docker Compose:** defines and orchestrates the entire multi-container environment of the system. Through a single `docker-compose.yml` file, it specifies the services (NiFi, HDFS namenode and datanode, Spark master and workers, Redis Stack, Grafana, Airflow), the dependencies between them, the internal network (`sabd_net`), and the persistent volumes, ensuring reproducibility and portability of the infrastructure.

III. DATA PIPELINE

A. Data Ingestion (Apache NiFi)

Apache NiFi manages the data acquisition process through the Process Group `sabd-ingest-flights-to-hdfs` (Figure 2). The NiFi flow automates the download of the monthly datasets in `.tar.gz` format, their decompression, and the loading of the CSVs onto HDFS under the path `hdfs://namenode:9000/data/raw/flights/csv/`.

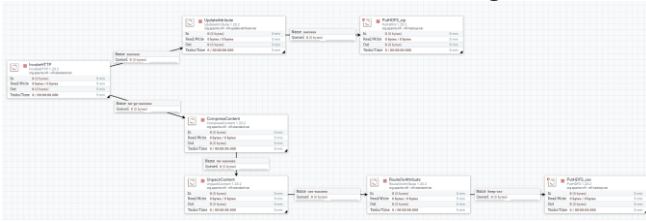


Figure 2 NiFi Dataflow - Data ingestion

The choice of NiFi over alternatives such as Apache Kafka or Pulsar is motivated by the batch nature of the process (4 monthly files, not a continuous stream): NiFi offers a graphical interface for flow configuration, native error handling and retry, and specialized processors for file and HDFS operations, without requiring the complexity of a distributed messaging system.

B. Preprocessing

The `preprocess.py` script reads the raw CSVs from HDFS applying an explicit schema and produces a processed Parquet dataset saved in `hdfs://namenode:9000/data/processed/flights/parquet`. The main operations include:

- Selection of the 14 columns needed for the three queries (reduction of the transferred volume).
- Explicit type casting: CANCELLED and DIVERTED converted from DoubleType to IntegerType.
- Removal of rows with a null `OP_UNIQUE_CARRIER` (invalid data with no identifiable airline).
- Imputation to 0 of null values in the delay cause columns (`CARRIER_DELAY`, `WEATHER_DELAY`, `NAS_DELAY`, `SECURITY_DELAY`, `LATE_AIRCRAFT_DELAY`): the BTS records these causes only when `ARR_DELAY > 15` minutes; treating the missing value as 0 preserves the consistency of the denominator in the computation of the averages.

The Parquet format was chosen over CSV, ORC, and Avro for its native support for column pruning (only the required columns are read from disk), its efficient compression, and its compatibility with Spark.

It was decided not to partition the dataset in Parquet format (for example, on a monthly basis), since the use of HDFS, natively optimized for large files, would have posed a threat to the performance of the pipeline.

The resulting dataset is a single logical Parquet managed by Spark through part-files.

C. Query Execution (Q1, Q2, Q3)

The three analytical queries were implemented using PySpark's DataFrame APIs as the main implementation. The DataFrame API leverages Spark's Catalyst optimizer, which generates optimized physical execution plans by automatically applying techniques such as column pruning, predicate pushdown, and shuffle optimization. In a second phase, each query was reimplemented using the low-level RDD APIs, which operate without automatic optimizations but offer explicit control over the data flow and serialization. The systematic comparison between the two implementations, carried out through a dedicated benchmark, takes place in Section V.

Q1 – Monthly statistics by airline

Q1 analyzes the flights of American Airlines (AA) and Delta Air Lines (DL) on a monthly basis, computing for non-cancelled flights: mean, minimum, and maximum of `DEP_DELAY` and `ARR_DELAY`; and for all flights in the group: cancellation rate (percentage of cancelled flights over the total).

Q1 - DataFrame:

1. Read Parquet file from HDFS (`spark.read.parquet`);
2. Select necessary columns (`select`);
3. Filter on target carriers AA and DL (`filter + isin`);
4. Aggregate by (month, airline) (`groupBy` on `MONTH`, `OP_UNIQUE_CARRIER`):
 - Count total flights, cancelled flights and non-cancelled flights (`F.count`, `F.sum`);
 - Mean, min and max of `DEP_DELAY` e `ARR_DELAY` on non-cancelled flights (`F.avg`, `F.min`, `F.max` with `F.when`);
5. Compute cancellation rate as a derived column (`withColumn`);
6. Round results and order results by company and month (`select`, `round`, `orderBy`);
7. Save on CSV format on HDFS and locally (`save_csv`).

The execution of the transformations is forced with `count` and `collect` (`save_csv`).

Q1 - RDD:

The query logic is identical to the DataFrame version. The main differences are:

- Conversion to RDD via `.rdd` after reading the Parquet, with caching of the result filtered on the target carriers;
- Three branches + join: the departure delay and arrival delay statistics (for which we consider only non-cancelled flights) and the cancellation rate statistics (computed instead on all flights) are calculated separately on three distinct branches, then rejoined via join on the key (month, carrier);
- Aggregation via `combineByKey` (for mean/min/max of `DEP_DELAY` and `ARR_DELAY`) and `reduceByKey` (for the cancellation rate counts). `combineByKey` was preferred over `groupByKey` because it performs partial aggregation within each partition before the shuffle, reducing the amount of data transferred between nodes.

Q2 - Top-10 Airlines by arrival delay

Q2 analyzes all airlines (non-cancelled and non-diverted flights with non-null `ARR_DELAY`), computing: mean `ARR_DELAY` and the means of the 5 delay components (`CARRIER_DELAY`, `WEATHER_DELAY`, `NAS_DELAY`, `SECURITY_DELAY`, `LATE_AIRCRAFT_DELAY`). A second phase filters the airlines with at least 500 valid flights and selects the top-10 by mean `ARR_DELAY` in descending order. The intermediate `DataFrame` of all airlines is cached, since it is reused for building the top-10.

Q2 - DataFrame:

1. Read Parquet file from HDFS (`spark.read.parquet`);
2. Select necessary columns (`select`);
3. Filter invalid flights: noncancelled, non diverted, `ARR_DELAY` not null (`filter`);
4. Aggregate by airline (`groupBy` `sum` `OP_UNIQUE_CARRIER`):
5. Count number of valid flights (`F.count`);
6. Mean `ARR_DELAY` and means of the five delay components (`F.avg`);
7. Rounding and alphabetical ordering of results (`select`, `round`, `orderBy`);
8. Cache materialization of the intermediate `DataFrame` (`cache`, `count`);
9. Filter airlines with at least 500 valid flights (`filter`);
10. Descending ordering of results by mean `ARR_DELAY` and selection of the first 10 results (`orderBy`, `limit`);
11. Save results in CSV format on HDFS and locally (`save_csv`).

Q2 - RDD:

The query logic is identical to the `DataFrame` version. The main differences concern:

- Conversion to RDD via `.rdd` after reading the Parquet;
- Aggregation via `combineByKey` (`create_combiner` / `merge_value` / `merge_combiners` pattern);
- Top-10 selection via `zipWithIndex` + filter.

- Release of the cached data via `unpersist`.

Q3 – Hourly Delay Percentiles

Q3 analyzes the non-cancelled flights with non-null `DEP_DELAY` of the airlines AA, DL, UA (United Airlines), and WN (Southwest Airlines), producing two outputs: (1) percentiles P25, P50, P75, P90 of `DEP_DELAY` for each (airline, hour slot 0-23) pair; (2) absolute min and max of `DEP_DELAY` per airline. Only non-cancelled flights with non-null `DEP_DELAY` and a valid hour (0-23) are included.

Q3 – DataFrame:

1. Reading of the Parquet file on HDFS (`spark.read.parquet`);
2. Projection of the necessary columns (`select`);
3. Filter on the target carriers, non-cancelled flights, and non-null `DEP_DELAY` (`filter`); derivation of the HOUR column from `CRS_DEP_TIME` via integer division by 100 (with `Column`, `cast`); filter on valid hours 0-23;
4. Materialization in cache of the filtered `DataFrame` (`cache`);
5. Output 1: aggregation by (airline, hour) (`groupBy`): flight count and percentiles P25/P50/P75/P90 via Spark's native function `F.percentile_approx` (Greenwald-Khanna algorithm, `accuracy=1000`, `relative error ≤ 0.1%`); sorting by airline and hour (`orderBy`);
6. Output 2: aggregation by airline (`groupBy`): absolute min and max of `DEP_DELAY` (`F.min`, `F.max`); sorting by airline (`orderBy`);
7. Saving of the two outputs in CSV format on HDFS and locally (`save_csv`).

Q3 – RDD:

The query logic is identical to the `DataFrame` version. The main differences concern:

- Conversion to RDD via `.rdd` after reading the Parquet;
- Percentiles: every partition builds a local `TDigest` using `t-digest [10]` via `mapPartitions`; the partial digests are aggregated between partitions with `reduceByKey` + `update_centroids_from_list`; the P25/P50/P75/P90 percentiles are extracted on the driver with `td.percentile()`;
- Min/Max: aggregation with `combineByKey`;

This approach avoids the collect of the raw data but introduces driver-executor communication overhead.

The choice of algorithms for Q3 follows a precise criterion: pairing each API with the most natural and representative algorithm for that programming model.

- **DataFrame + percentile_approx (Greenwald-Khanna).** `F.percentile_approx` is Spark's native function for the approximate computation of percentiles on `DataFrames`. Internally it adopts the Greenwald-Khanna algorithm, which builds a distributed summary directly on the executors without ever transferring data to the driver. It is the idiomatic approach for large datasets with the `DataFrame` API: a single declarative call, fully optimized by the Catalyst optimizer.

- **RDD + t-digest.** T-digest is an alternative algorithm based on variable-density centroids, more accurate at the tails of the distribution, where extreme percentiles such as P90 are harder to estimate, and naturally composable: each partition builds its own local digest, and the digests are then merged via `reduceByKey + update_centroids_from_list`.

The two methods produce numerically close but semantically different results. `percentile_approx` implements a discrete empirical quantile and always returns a value observed in the dataset, a behavior appropriate for variables such as `DEP_DELAY` expressed in whole minutes. T-digest performs linear interpolation between the centroids of the sketch, returning decimal values that do not correspond to real observations. From the standpoint of precision, the maximum difference observed between the two methods is less than 1 minute across all the analyzed quantiles, confirming that both approaches are adequate for this type of analysis.

D. Result Export (Redis)

We chose to use the NoSQL key-value datastore Redis, specifically in the Redis Stack distribution (7.2.0-v10), as the serving layer for the subsequent visualization of the output in Grafana dashboards.

Redis Stack extends Redis with additional modules and advanced data structures, including `RedisTimeSeries` and `RedisJSON`, making it more suitable for storing and querying structured results compared to a plain Redis instance.

The main motivations for this choice are related to the flexibility of the key-value model offered by Redis, suitable for storing small aggregated results, such as statistics by month, airline, and hour. Furthermore, `RedisTimeSeries` was used to represent data ordered along a time axis, for example months or hours of the day (*Query 1* and *Query 3*), while Hashes and Lists were used to represent tables, indexed metrics, and sorted rankings. A further element in favor of this choice was the availability of the free Redis Datasource plugin for Grafana, which allows Redis to be queried directly from the dashboards.

The use of Redis as the serving layer required a careful choice of keys that would allow adequate visualization in Grafana. The mapping from CSV to key-value takes place in the two export files `export_output_to_redis.py` and `export_benchmark_to_redis.py`.

E. Visualization (Grafana)

For the visualization, Grafana 10.4.0 was used, together with the `redis-datasource` plugin.

Three main dashboards were created, one for each query. To build the individual panels, the data are retrieved from the redis datasource through ad hoc queries, using Redis commands compatible with the data structures defined in the export phase (`TS.RANGE`, `HGET`, etc.), and subsequently manipulated through the application of appropriate transformations (`extract`, `organize`, `join`, `convert`, etc.).

The persistence of the Grafana configuration is managed through provisioning in the corresponding project folders. In particular, both the Redis datasource and the dashboards are loaded automatically at the startup of the Grafana container, making the visualization environment reproducible.

F. Orchestration Pipeline (Apache AirFlow)

The `sabd_project1_pipeline` DAG in Airflow coordinates the entire flow following the pipeline (*Figure 3*).

Airflow is configured with SQLite as the backend and `SequentialExecutor`, which executes the tasks sequentially even when they are defined as parallel in the DAG. This choice is motivated by the execution context: the entire architecture runs on WSL2, an environment in which the available memory and CPU resources are limited and shared among all the Docker containers. Running the three Spark queries simultaneously would have generated contention on the executor resources, risking timeouts or failures due to memory exhaustion. Sequential execution ensures that each query has the full capacity of the Spark cluster available during its own execution, at the cost of a time overhead that is acceptable in the batch context of the project.



Figure 3. Airflow DAG

IV. RESULTS

The following are the results of the three queries, starting from *Query 1*.

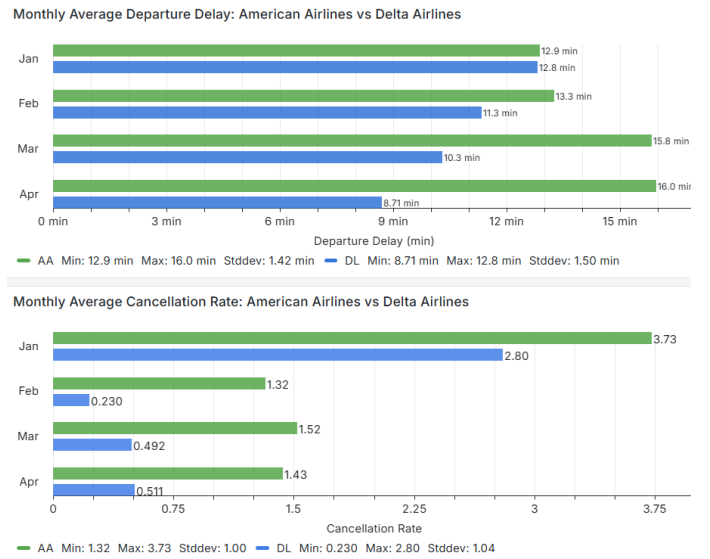


Figure 4. Results of *Query 1*

The results of *Query 2* are reported below: ranking of the top ten airlines by mean arrival delay and mean contributions of the different delay causes.

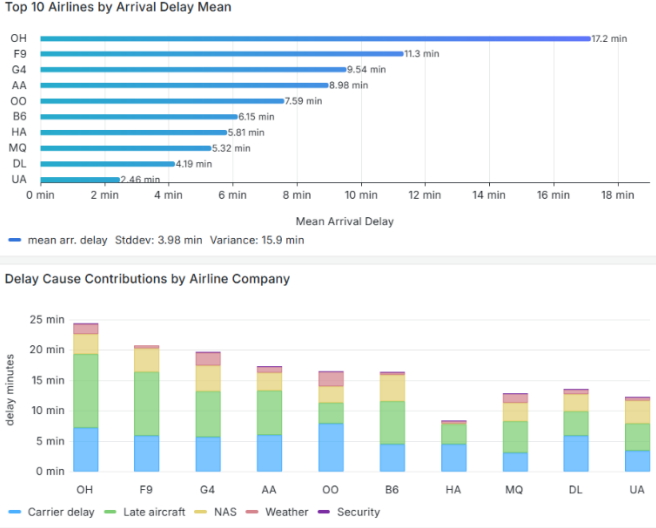


Figure 5 Results of Query 2

Finally, each of the following panels illustrates the percentiles (25th, 50th, 75th, 90th) for each of the 4 target companies.



Figure 6. Results of Query 3

V. PERFORMANCE ANALYSIS

To analyze the performance of the system and the quality of the design choices, four experiments were carried out, whose

outcomes made it possible to identify the optimal configuration for the chosen pipeline and deployment. The first experiment concerns the performance comparison between the DataFrame format and the RDD format. The second experiment measures performance as the number of Spark Workers varies. The third experiment investigates the impact of Spark Session caching, comparing "Cold Start" runs, where the Spark Session is recreated each time, and "Warm Start" runs, where the Spark Session is shared across iterations. Finally, the last experiment evaluates the impact of manual caching on query performance. All experiments were carried out in local deployment on the same host machine, which has an AMD Ryzen 7 5700U Series with Radeon Graphics processor, 8 cores, and 16GB of RAM.

A. Performance metrics

For the performance evaluation, the following main metrics were defined:

- **Loading time:** time to read the Parquet data from HDFS, including the `select` operation used to force the actual loading of the data.
- **Processing time:** pure query computation time; it corresponds to the `computation_s` phase and represents the pure cost of the Spark transformations.
- **End-to-end time:** total execution time (from start to finish), including Spark setup, I/O, and system overhead.

Timings were collected paying attention to Spark's lazy evaluation mechanism: to measure the time of a phase, the difference is computed between the instant preceding the transformations and the instant immediately following the first useful action that forces their execution.

B. DataFrame vs RDD

The performance comparison between the implementations based on DataFrame APIs and those developed using RDDs was conducted on all three analytical queries, using a single Spark Worker. The end-to-end experimental results, summarized in Table 1, highlight a clear and structural discrepancy in the measured times, underscoring the advantages of the high-level abstraction (which leverages the Catalyst Optimizer) over the low-level one.

Table 1: DataFrame vs RDD (end-to-end time)

Query	API	Mean	Std. Dev.
Q1	DataFrame	2.91	0.53
Q1	RDD	40.09	1.78
Q2	DataFrame	3.38	0.39
Q2	RDD	62.70	0.76
Q3	DataFrame	18.78	1.19
Q3	RDD	24.27	0.31

Table 2: DataFrame vs RDD (processing time)

Query	API	Mean	Std. Dev.
Q1	DataFrame	0.68	0.13
Q1	RDD	38.95	1.82
Q2	DataFrame	1.06	0.10
Q2	RDD	61.26	0.81
Q3	DataFrame	4.32	0.75
Q3	RDD	21.95	0.29

C. Worker Scalability

For the purposes of the scalability analysis, the benchmark was run with a variable number of workers, from a minimum of 1 to a maximum of 4. To allow local execution, the Spark Workers were limited to 1GB of memory and 1 core (`docker_compose.yml`). For each configuration (query; format; number of workers), 20 iterations were performed, of which 5 were warm-up runs excluded from the statistics and 15 were considered valid. The analysis of the results (*Tables 3 and 4, Figures 6, 7, and 8, 9*) shows that, for the chosen configuration, the use of 2 workers represents the best trade-off between execution time and resources employed. This conclusion derives from the trend of the scaling curves: moving from 1 to 2 workers produces a significant reduction in times, while adding further workers does not yield proportional improvements and, in some cases, introduces a significant degradation in performance (*Figure 8, RDD API*).

Table 3: Results of the scaling analysis – End-to-End Time

Query	API	1W	2W	3W	4W
Q1	DataFrame	2.91 ± 0.53	1.73 ± 0.21	1.70 ± 0.32	2.07 ± 0.28
Q1	RDD	40.09 ± 1.78	9.49 ± 0.68	14.32 ± 1.20	61.88 ± 157.30
Q2	DataFrame	3.38 ± 0.39	1.42 ± 0.17	1.53 ± 0.16	1.63 ± 0.29
Q2	RDD	62.70 ± 0.76	12.92 ± 0.56	12.71 ± 0.62	12.63 ± 0.67
Q3	DataFrame	18.78 ± 1.19	4.58 ± 0.24	4.75 ± 0.53	5.27 ± 0.70
Q3	RDD	24.27 ± 0.31	15.74 ± 1.77	15.23 ± 1.07	15.59 ± 0.77

Shown values are presented as mean ± standard deviation, computed on the total of 15 valid iterations. 1W = 1 Worker.

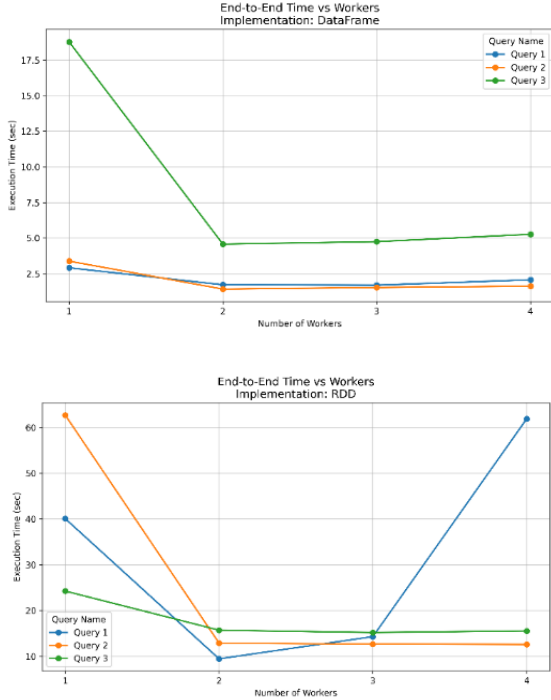


Figure 7. End-to-end time comparison (Dataframe vs RDD)

Table 4: Results of the scaling analysis – Processing Time

Query	API	1W	2W	3W	4W
Q1	DataFrame	0.68 ± 0.13	0.46 ± 0.04	0.46 ± 0.09	0.57 ± 0.09
Q1	RDD	38.95 ± 1.82	8.83 ± 0.34	13.71 ± 1.12	60.99 ± 157.13
Q2	DataFrame	1.06 ± 0.10	0.48 ± 0.04	0.56 ± 0.05	0.51 ± 0.04
Q2	RDD	61.26 ± 0.81	12.04 ± 0.39	11.66 ± 0.61	11.32 ± 0.61
Q3	DataFrame	4.32 ± 0.75	0.86 ± 0.06	0.91 ± 0.14	1.06 ± 0.19
Q3	RDD	21.95 ± 0.29	13.57 ± 1.01	13.23 ± 1.04	13.17 ± 0.75

Shown values are presented as mean ± standard deviation, computed on the total of 15 valid iterations. 1W = 1 Worker.

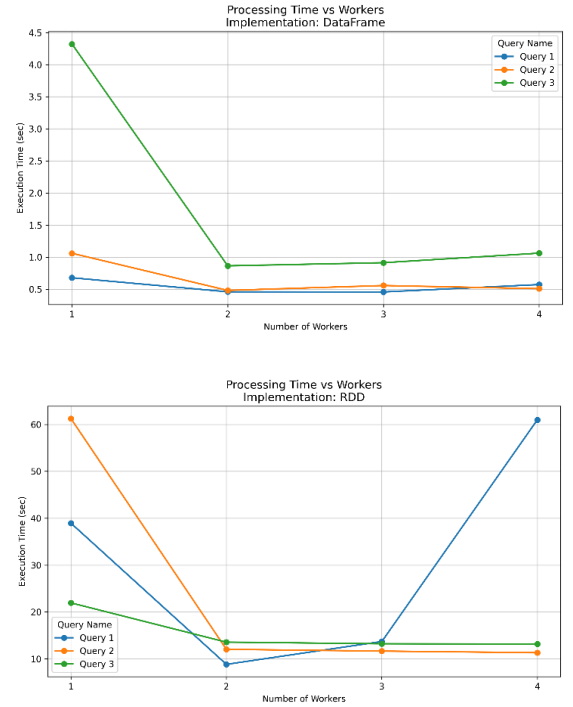


Figure 8. Processing time comparison (Dataframe vs RDD)

D. Warm start vs Cold start

To evaluate the impact of Spark Session caching, two different benchmarks were carried out:

- **Warm start benchmark:** 20 total iterations per query and per implementation, of which the first 5 are warm-up runs excluded from the statistics. The 15 valid iterations separately measure the four phases (`loading_s`, `computation_s`, `output_s`, `end_to_end_s`). The Spark session is shared across iterations: starting from the first runs, the Parquet is already cached at the OS/JVM level, simulating a realistic operational workload. The CSV write is performed at every valid iteration to give `output_s` the same statistical consistency as the other phases.
- **Cold start benchmark:** 15 total iterations per query, with no warm-up iterations. The Spark session is terminated and recreated between one iteration and the next, resetting every form of in-memory caching. This

scenario simulates the execution of the pipeline under cold-start conditions, measuring the real cost of the first access to the data.

The results in Table 5 refer to the execution of the benchmark with 1 Spark Worker and the DataFrame API.

Table 5: Benchmark results: Cold Start vs Warm Start

Query	Phase	Mode	Mean (s)	StdDev
Q1	Spark start	cold start	0.70	1.33
Q1	Loading	cold start	14.60	1.75
Q1	Loading	warmup	0.55	0.04
Q1	Processing	cold start	6.16	0.95
Q1	Processing	warmup	0.77	0.07
Q1	End-to-end	cold start	25.37	4.43
Q1	End-to-end	warmup	2.86	0.45
Q1	Spark stop	cold start	0.39	0.14
Q2	Spark start	cold start	0.33	0.03
Q2	Loading	cold start	13.94	0.36
Q2	Loading	warmup	0.45	0.03
Q2	Processing	cold start	7.73	0.31
Q2	Processing	warmup	0.79	0.07
Q2	End-to-end	cold start	26.36	0.59
Q2	End-to-end	warmup	2.83	0.20
Q2	Spark stop	cold start	0.43	0.12
Q3	Spark start	cold start	0.32	0.02
Q3	Loading	cold start	14.20	0.43
Q3	Loading	warmup	0.39	0.04
Q3	Processing	cold start	15.20	0.55
Q3	Processing	warmup	8.05	0.19
Q3	End-to-end	cold start	41.41	1.19
Q3	End-to-end	warmup	15.14	0.36
Q3	Spark stop	cold start	0.30	0.09

The results show a significant impact of warm-up on all execution phases. The most marked difference concerns loading: in cold start, reading the Parquet takes about 3.5s for all queries, while in warm session it drops to 0.06-0.09s, confirming that after the first iterations the file is kept in the OS/JVM cache. Processing also improves noticeably thanks to the JVM's JIT compilation, which optimizes the bytecode at runtime after identifying the hottest code paths (hotspots). The variability of the measurements is considerably higher in cold start, reflecting the instability of the first disk access and of the JVM warm-up. In warm session the variability is drastically reduced, confirming the stability of the measurements in the valid iterations.

E. Impact of caching

A final observation concerns the impact of the developer's use of `.cache()` within the queries, since it affects both the measurement of the times and the actual behavior. Applying the cache to every intermediate DataFrame makes it possible to separate loading, computation, and output more sharply, but it introduces additional materializations, increases memory pressure, and can reduce Spark's freedom to optimize the plan (Catalyst Optimizer). In the tests, this translates into higher times in the initial phases, which also include the cost of populating the cache, and lower processing times because they are based on already materialized data.

To avoid measuring an artificial version of the query, caching was therefore kept only where justified by an actual reuse of the DataFrame, for example when a result is used by multiple actions or by multiple outputs. The final methodology thus favors more realistic times, accepting a less perfect separation of the phases that is, however, more representative of real Spark execution.

Table 6: Impact of caching – Warm Session

Query	Phase	Impl.	Mean (s)	StdDev
Q1	Loading	cache	2.36	0.23
Q1	Loading	no cache	0.55	0.04
Q1	Processing	cache	0.43	0.06
Q1	Processing	no cache	0.77	0.07
Q1	Output	cache	0.65	0.25
Q1	Output	no cache	0.89	0.36
Q1	End-to-end	cache	4.37	0.53
Q1	End-to-end	no cache	2.86	0.45

*Cache = caching in each intermediate phase
No cache = caching only where necessary (baseline)*

VI. CONCLUSIONS

The experimental results confirm the superiority of the DataFrame API over the RDD API for all three analyzed queries. This advantage is mainly attributable to the Catalyst optimizer, which automatically plans and executes the query plan efficiently, unlike the RDD API, which does not benefit from such optimizations and introduces additional overhead related to serialization and the explicit management of transformations.

Regarding scaling, the configuration with a reduced number of workers proved to be optimal in the considered environment. This result is strictly dependent on the deployment context (standalone node on Docker Compose and WSL2 with limited and shared resources), where the inter-container coordination overhead tends to reduce the gains deriving from greater parallelization. Therefore, the scaling results could differ significantly on a physical multi-node infrastructure.

The effect of Spark caching on performance turned out to be marked, in particular on the `computation_s` metric, consistently with the fundamental design of the framework, which reduces the cost of recomputations by keeping intermediate datasets in RAM.

Overall, the project shows how a batch pipeline based on NiFi, HDFS, Spark, Redis, and Grafana can be built end-to-end in a containerized environment, covering ingestion, preprocessing, analytical queries, benchmarking, result serving, and visualization. From a performance standpoint, the experiment highlights the advantage of using Spark's declarative APIs over more manual RDD-based implementations, but also the importance of the infrastructural context in the evaluation of scaling. Future developments could include running the benchmarks on a real distributed cluster and further performance analyses.

End-to-end execution time distribution

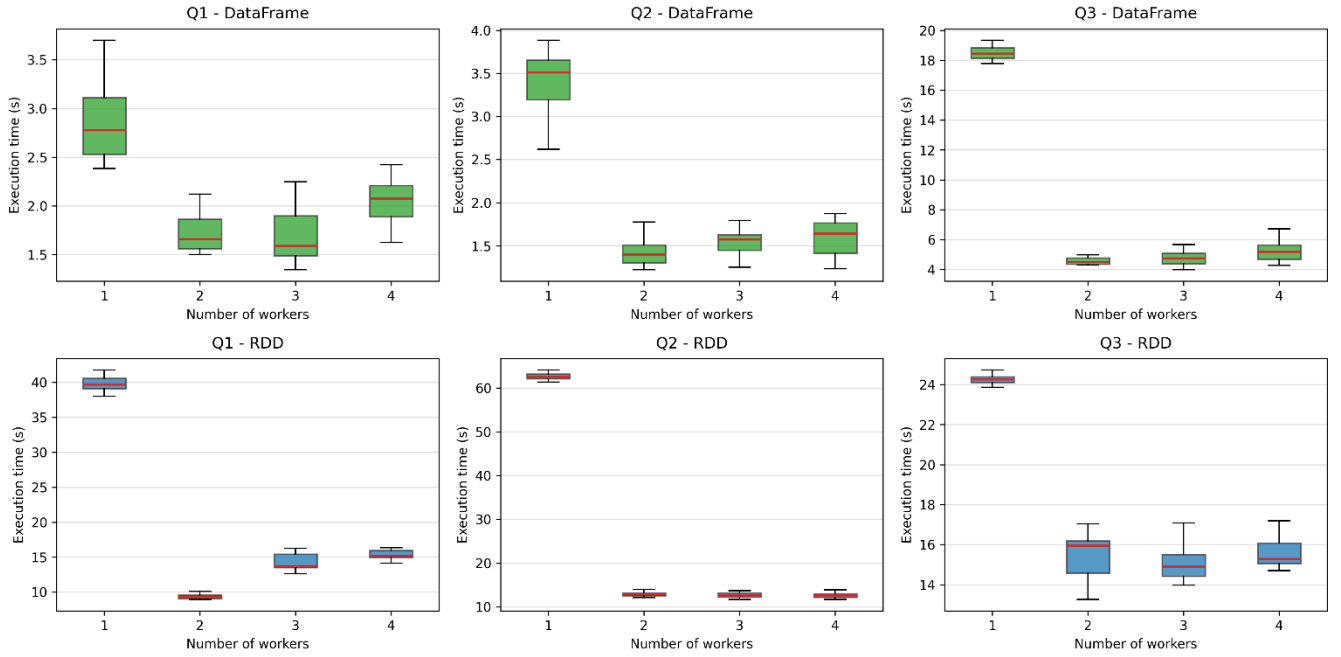


Figure 9. End to End time by query. *Note: outliers are not shown for readability.*

Processing time distribution

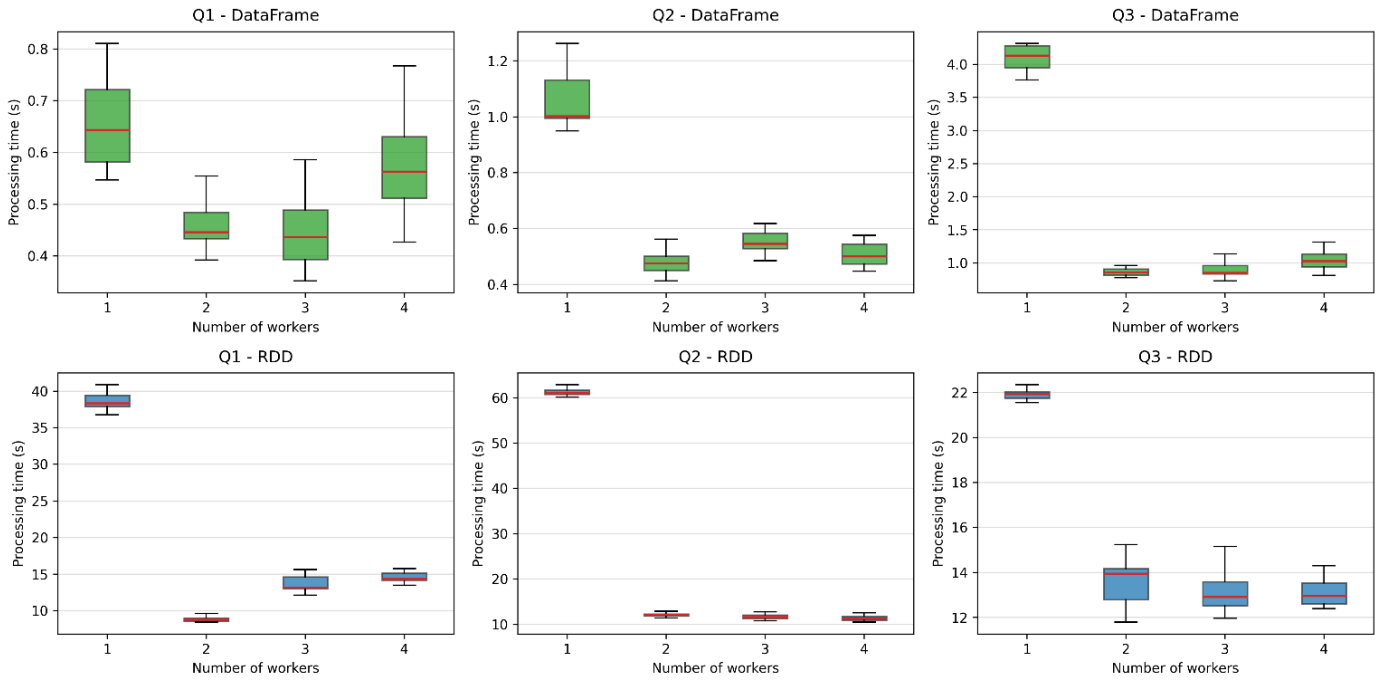


Figure 10. Processing time by query. *Note: outliers are not shown for readability.*

REFERENCES

- [1] Apache Software Foundation. Apache Spark. Available: <https://spark.apache.org/>
- [2] U.S. Department of Transportation. Bureau of Transportation Statistics (BTS). Available: <https://www.transtats.bts.gov/>
- [3] Docker. Available: <https://www.docker.com/>
- [4] Apache Software Foundation. Apache Airflow. Available: <https://airflow.apache.org/>
- [5] Apache Software Foundation. Apache NiFi. Available: <https://nifi.apache.org/>
- [6] Apache Software Foundation. Hadoop Distributed File System (HDFS). Available: <https://hadoop.apache.org/>
- [7] Redis. Available: <https://redis.io/>
- [8] Grafana Labs. Grafana: The open observability platform. Available: <https://grafana.com/grafana/>
- [9] Redis Stack. <https://redis.io/about/redis-stack/>
- [10] T. Dunning. The t-digest: Efficient estimates of distributions. *Software Impacts*, 7:100049, 2021. <https://www.sciencedirect.com/science/article/pii/S2665963820300403>